

EXPERIMENT - 6

Aim

Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory

A **Smart Contract** is a **self-executing digital program stored on a blockchain** that automatically performs actions when predefined conditions are met. The concept was first proposed by **Nick Szabo** in the 1990s. The goal of smart contracts is to allow agreements between parties to be executed **automatically, securely, and without intermediaries**.

In traditional agreements, a third party such as a bank, lawyer, or broker is often required to verify and enforce the terms of a contract. Smart contracts remove the need for such intermediaries by embedding the **rules and conditions directly into computer code**. When the specified conditions are fulfilled, the contract automatically executes the agreed-upon action.

Smart contracts are widely used on blockchain platforms such as **Ethereum**. They are typically written in programming languages such as **Solidity** and executed on the **Ethereum Virtual Machine (EVM)**. The EVM ensures that the smart contract code runs exactly as programmed across all nodes in the blockchain network.

Once deployed to the blockchain, a smart contract becomes **immutable**, meaning its code cannot easily be altered. Every transaction or execution related to the contract is recorded on the blockchain, providing a **permanent and transparent record** of activities.

Smart contracts enable **trustless interactions**, meaning participants do not need to rely on trust in each other or a central authority. Instead, they rely on the correctness and transparency of the code and the blockchain network.

Significance of Smart Contracts in the Ethereum Blockchain

1. Automation

Smart contracts automatically execute predefined actions when certain conditions are met.

For example, if a buyer sends payment, the smart contract can automatically transfer ownership of a digital asset without manual intervention.

2. Transparency

The contract code and transaction history are visible on the blockchain network. Anyone can verify the logic of the contract and check how it operates, which increases accountability and reduces fraud.

3. Security

Smart contracts are stored on the blockchain, which uses cryptographic techniques to secure data. Once deployed, the contract becomes **tamper-resistant and immutable**, preventing unauthorized modifications.

4. Cost Efficiency

Since smart contracts eliminate intermediaries such as brokers, lawyers, or financial institutions, they significantly reduce transaction costs and administrative overhead.

5. Trustless Interaction

Participants do not need to trust each other or rely on a third party. The contract executes strictly according to the programmed rules, ensuring fairness and reliability.

6. Faster Transactions

Traditional contract processing may take days due to paperwork and verification processes. Smart contracts can execute transactions **almost instantly** once the required conditions are satisfied.

Role of Smart Contracts in Modern Blockchain Applications

Smart contracts form the **core infrastructure of decentralized applications (DApps)**. They are also widely used in **Decentralized Finance (DeFi)** platforms, where they enable services such as lending, borrowing, trading, and insurance without traditional banks.

Popular DeFi platforms such as **Uniswap** and **Aave** rely heavily on smart contracts to operate automatically and securely.

Implementation:

The project “Land Ownership Registry System” focuses on automating land records and ownership transfers using Ethereum-based Smart Contracts.

To ensure the system works, we will implement two primary contracts:

1. **LandRegistry.sol**: Manages the database of lands and their owners.
2. **LandTransfer.sol**: Handles the logic of transferring ownership from a Seller to a Buyer.

Code:-**1. LandRegistry.sol**

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract LandRegistry {
    struct Land {
        uint id;
        string location;
        uint area;
        address owner;
        bool isRegistered;
    }

    mapping(uint => Land) public lands;
    address public admin;

    constructor() {
        admin = msg.sender;
    }

    function registerLand(uint _id, string memory _location, uint _area, address _owner)
    public {
        require(msg.sender == admin, "Only admin can register land");
        require(!lands[_id].isRegistered, "Land already registered");

        lands[_id] = Land(_id, _location, _area, _owner, true);
    }

    function updateOwner(uint _id, address _newOwner) public {
        // This function will be called by the Transfer contract
        lands[_id].owner = _newOwner;
    }

    function getLand(uint _id) public view returns (uint, string memory, uint, address) {
        Land memory l = lands[_id];

        return (l.id, l.location, l.area, l.owner);
    }
}
```

2.LandTransfer.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "./LandRegistry.sol";

contract LandTransfer {
    LandRegistry public registry;

    constructor(address _registryAddress) {
        registry = LandRegistry(_registryAddress);
    }

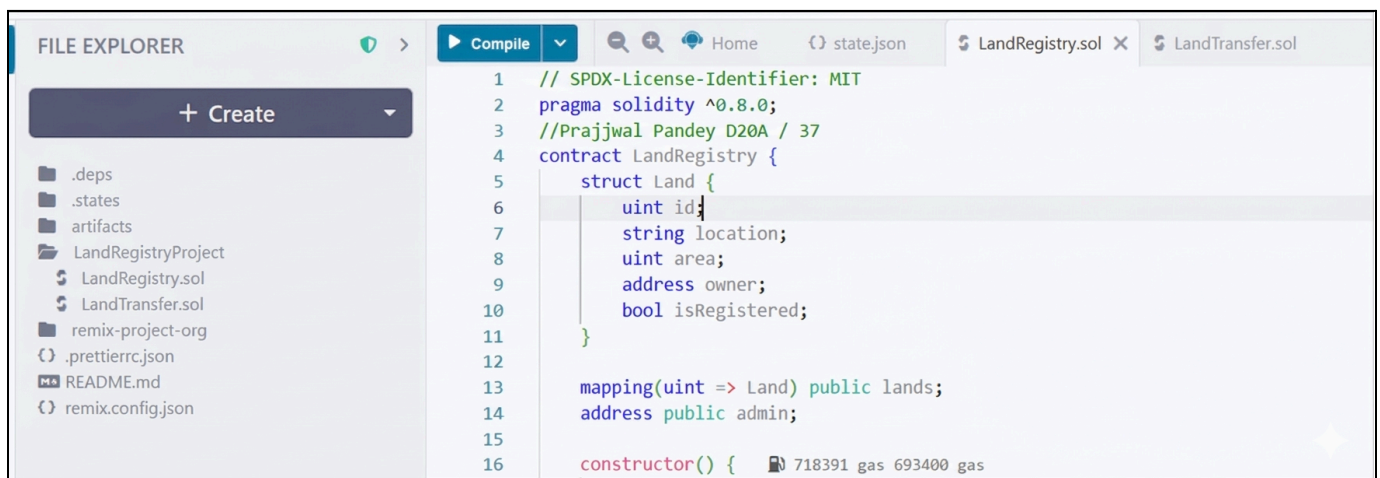
    function transferOwnership(uint _landId, address _newOwner) public {
        (,, address currentOwner) = registry.getLand(_landId);

        require(msg.sender == currentOwner, "Only the owner can sell this land");

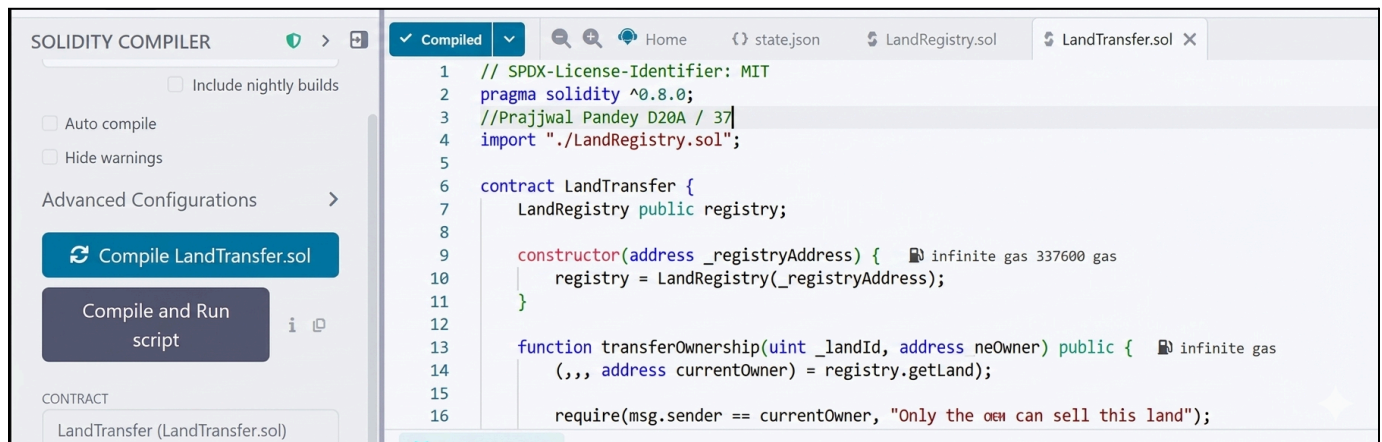
        registry.updateOwner(_landId, _newOwner);
    }
}
```

Step 1: Set up the Files

1. Go to Remix IDE.
2. In the File Explorer, create a new folder named LandRegistryProject.
3. Create two files: LandRegistry.sol and LandTransfer.sol.



Step 2: Compile both the contracts



SOLIDITY COMPILER

Include nightly builds

☐ Auto compile

☐ Hide warnings

Advanced Configurations

Compile LandTransfer.sol

Compile and Run script

CONTRACT

LandTransfer (LandTransfer.sol)

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3 //Prajwal Pandey D20A / 37
4 import "./LandRegistry.sol";
5
6 contract LandTransfer {
7     LandRegistry public registry;
8
9     constructor(address _registryAddress) {
10         registry = LandRegistry(_registryAddress);
11     }
12
13     function transferOwnership(uint _landId, address newOwner) public {
14         (,,, address currentOwner) = registry.getLand(_landId);
15
16         require(msg.sender == currentOwner, "Only the owner can sell this land");
```



SOLIDITY COMPILER

0.8.31+commit.f3a2265

Include nightly builds

☐ Auto compile

☐ Hide warnings

Advanced Configurations

Compile LandTransfer.sol

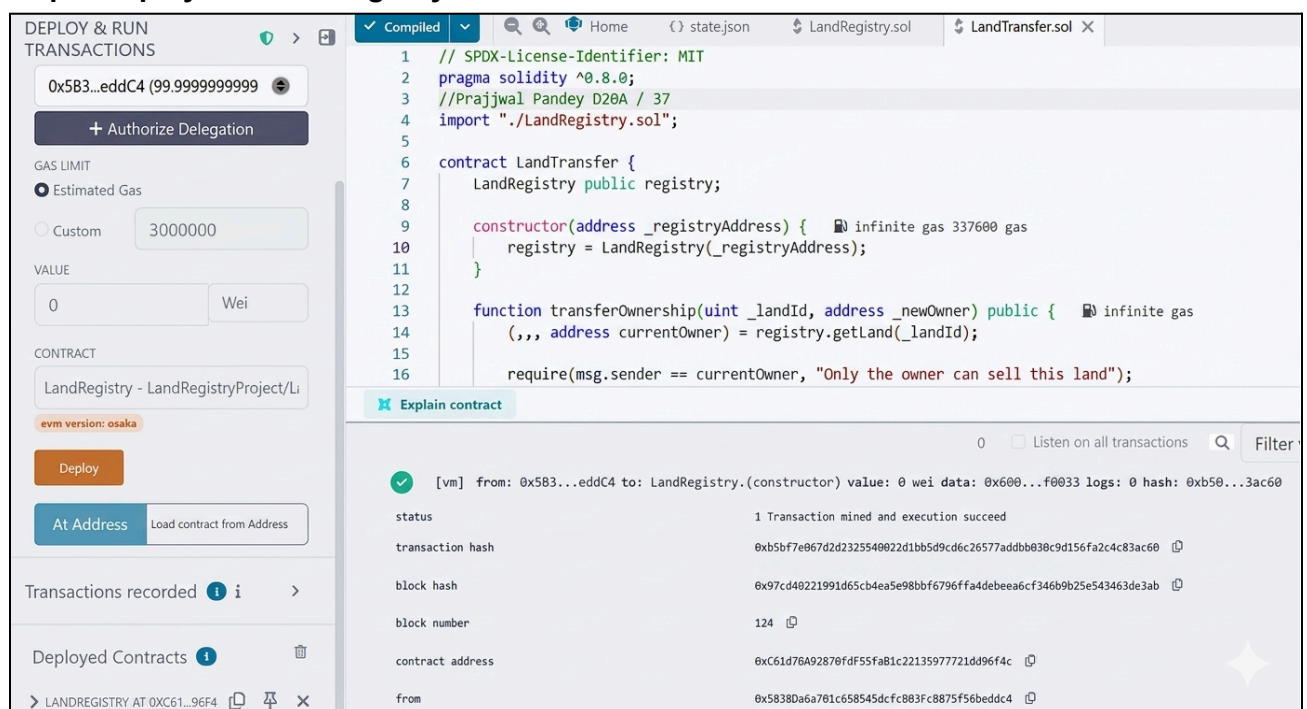
Compile and Run script

CONTRACT

LandTransfer (LandTransfer.sol)

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3 //Prajwal Pandey D20A / 37
4 import "./LandRegistry.sol";
5
6 contract LandTransfer {
7     LandRegistry public registry;
8
9     constructor(address _registryAddress) {
10         registry = LandRegistry(_registryAddress);
11     }
12
13     function transferOwnership(uint _landId, address newOwner) public {
14         (,,, address currentOwner) = registry.getLand(_landId);
15
16         require(msg.sender == currentOwner, "Only the owner can sell this land");
```

Step 3: Deploy the LandRegistry contract



DEPLOY & RUN TRANSACTIONS

0x583...eddC4 (99.999999999)

+ Authorize Delegation

GAS LIMIT

☒ Estimated Gas

☐ Custom 3000000

VALUE

0 Wei

CONTRACT

LandRegistry - LandRegistryProject/Li

evm version: osaka

Deploy

At Address Load contract from Address

Transactions recorded

Deployed Contracts

LANDREGISTRY AT 0XC61...96F4

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3 //Prajwal Pandey D20A / 37
4 import "./LandRegistry.sol";
5
6 contract LandTransfer {
7     LandRegistry public registry;
8
9     constructor(address _registryAddress) {
10         registry = LandRegistry(_registryAddress);
11     }
12
13     function transferOwnership(uint _landId, address newOwner) public {
14         (,,, address currentOwner) = registry.getLand(_landId);
15
16         require(msg.sender == currentOwner, "Only the owner can sell this land");
```

Explain contract

0 Listen on all transactions Filter

[vm] from: 0x583...eddC4 to: LandRegistry.(constructor) value: 0 wei data: 0x600...f0033 logs: 0 hash: 0xb50...3ac60

status 1 Transaction mined and execution succeed

transaction hash 0xb5bf7e067d2d23255400221bb5d9c6c26577adbb030c9d156fa2c4c83ac60

block hash 0x97cd40221991d65cb4ea5e98bbf6796ffa4debea6cf346b9b25e543463de3ab

block number 124

contract address 0xc61d76A92870fDf55Fa81c2213597721d096f4c

from 0x5838D0a6a781c658545dcfc883fc8875f56beddc4

Step 4: Copy the contract address of the deployed LandRegistry contract

15
16

require(msg.sender == currentOwner, "Only the owner can sell this land");

 Explain contract

0 ☐ Listen on all transactions 

transaction hash	0xb50f7e067d2d9329540023d1b65d9cd6c265f7adbb030c9d156fa2c4c83ac60 
block hash	0x97cd49221691d65cb4ea5e98bbf6798ffa4debeea6cf346b9b25e943463de3ab 
block number	124 
contract address	0xC61d78A9287DfdF85fAB1c22135977721dd96F4c 
from	0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 
to	LandRegistry.(constructor) 
transaction cost	824515 gas 

Step 5: Select LandTransfer here in the dropdown

GAS LIMIT

☒ Estimated Gas

☐ Custom 3000000

VALUE

0 Wei

CONTRACT

LandTransfer - LandRegistryProject/Li

LandRegistry - LandRegistryProject/LandRegistry.sol

LandTransfer - LandRegistryProject/LandTransfer.sol

At Address

Load contract from Address

Transactions recorded 2 i >

Deployed Contracts 2

> LANDREGISTRY AT 0XC61...96F4   

> LANDTRANSFER AT 0XA2E...DC1   

4 import './LandRegistry.sol';

5

6 contract LandTransfer {

7 LandRegistry public registry;

8

9 constructor(address _registryAddress) {  infinite gas 337600 gas

10 | registry = LandRegistry(_registryAddress);

11 }

12

13 function transferOwnership(uint _landId, address _newOwner) public {  infinite gas

currentOwner = registry.getLand(_landId);

sender == currentOwner, "Only the owner can sell this land");

 Explain contract

0 ☐ Listen on all transactions  Filter with tra

creation of LandRegistry pending...

 [vm] from: 0x5B3...eddC4 to: LandRegistry.(constructor) value: 0 wei data: 0x608...f0033 logs: 0 hash: 0xb50...3ac60

status 1 Transaction mined and execution succeed

transaction hash 0xb50f7e067d2d9329540023d1b65d9cd6c265f7adbb030c9d156fa2c4c83ac60 

block hash 0x97cd49221691d65cb4ea5e98bbf6798ffa4debeea6cf346b9b25e943463de3ab 

block number 124 

contract address 0xC61d78A9287DfdF85fAB1c22135977721dd96F4c 

Step 6: Add the contract address in place of `_registryAddress` field next to the deploy button, and click on deploy so as to deploy the LandTransfer contract

DEPLOY & RUN
TRANSACTIONS

+ Authorize Delegation

GAS LIMIT

☒ Estimated Gas

☐ Custom

3000000

VALUE

0

Wei

CONTRACT

LandTransfer - LandRegistryProject/La

evm version: osaka

Deploy

0xC61d78A92B7DfdF85fAB1c22

At Address

Load contract from Address

[vm] from: 0x5B3...eddC4 to: LandTransfer.(constructor) value: 0 wei data: 0x608...96f4c logs: 0 hash: 0x02b...545c1

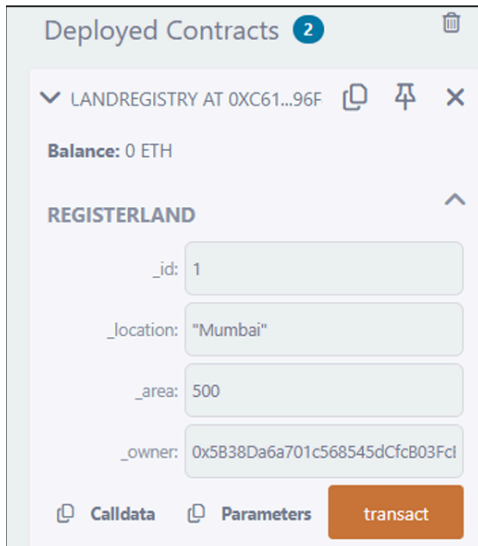
Debug

status	1 Transaction mined and execution succeed
transaction hash	0x02bfc4bb38722b91c6980f77829c9fe52aa29f2f6e96765034aa9b1a3f4545c1 🔗
block hash	0x30d868035e979b6362cde68e0892bca6d6a6f80774dc8473ada8cb5da89296bb 🔗
block number	125 🔗
contract address	0xa2e9669fC58055D0aF18aEd20dcF10A9e00Cb97 🔗
from	0x5B380a6a701c568545dcFc803Fc8875f56beddC4 🔗
to	LandTransfer.(constructor) 🔗
transaction cost	443826 gas 🔗
execution cost	360488 gas 🔗
output	0x608060d09573d801561000f575f5ff45h50600d3610061003d575f3560e01c806379507f731d6100385780637h1030901d61005d575h5f5ff45h610057600d4803603810

Step 7: Execute the transactions

1.Land Registration

In the deployed contracts section expand the LandRegistry contract, and add details as mentioned. Also ensure that the wallet address is added in the owner field. Once all this is done, click on transact to complete the transaction.



Deployed Contracts 2

LANDREGISTRY AT 0XC61...96F

Balance: 0 ETH

REGISTERLAND

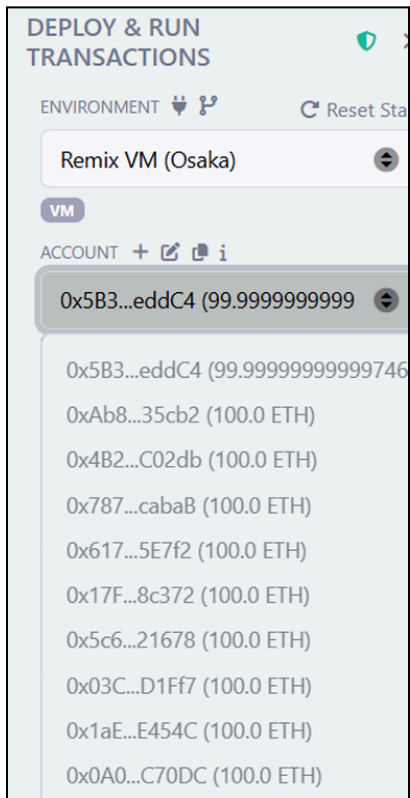
_id: 1

_location: "Mumbai"

_area: 500

_owner: 0x5B38Da6a701c56854dCfcB03Fcl

Calldata Parameters transact



DEPLOY & RUN TRANSACTIONS

ENVIRONMENT Reset State

Remix VM (Osaka)

VM

ACCOUNT +

0x5B3...eddC4 (99.9999999999)

0x5B3...eddC4 (99.9999999999746)

0xAb8...35cb2 (100.0 ETH)

0x4B2...C02db (100.0 ETH)

0x787...cabaB (100.0 ETH)

0x617...5E7f2 (100.0 ETH)

0x17F...8c372 (100.0 ETH)

0x5c6...21678 (100.0 ETH)

0x03C...D1Ff7 (100.0 ETH)

0x1aE...E454C (100.0 ETH)

0x0A0...C70DC (100.0 ETH)

The screenshot displays the 'Transactions recorded' section in the Remix IDE. On the left, the 'Deployed Contracts' panel shows the 'LANDREGISTRY AT 0xC61...96F' contract with a balance of 0 ETH. The 'REGISTERLAND' function is expanded, showing input fields for _id (1), _location (Mumbai), _area (500), and _owner (0x5B38Da6a701c56854dCfcB03Fc8875f56beddC4). The 'transact' button is visible. The main panel shows the transaction details for the 'registerLand' function, including the 'from' address (0x5B38Da6a701c56854dCfcB03Fc8875f56beddC4), the 'to' address (LandRegistry.registerLand), the transaction cost (115835 gas), the execution cost (93631 gas), the output (0x), the decoded input (a JSON object with _id, _location, _area, and _owner), the decoded output ({}), and the logs ([]).

Enter id=1 in the getLand field to see the registered data

The screenshot displays the 'Low level interactions' section in the Remix IDE. The 'getLand' function is selected, and the input field is set to 1. The output of the function is shown as a JSON object: { "0": "uint256: 1", "1": "string: Mumbai", "2": "uint256: 500", "3": "address: 0x5B38Da6a701c56854dCfcB03Fc8875f56beddC4" }. The 'lands' dropdown is set to 'uint256'. The 'Low level interactions' panel shows the 'getLand' function call and its output.

2. Transfer Ownership

Copy a different wallet address from the account dropdown (this wallet address represents the buyer)

The screenshot displays the 'ACCOUNT' dropdown menu in the Remix IDE. The menu is open, showing a list of wallet addresses and their balances. The selected address is '0xAb8...35cb2 (100.0 ETH)'. Other addresses include '0x5B3...eddC4 (99.9999999999723)', '0x4B2...C02db (100.0 ETH)', '0x787...cabaB (100.0 ETH)', '0x617...5E7f2 (100.0 ETH)', '0x17F...8c372 (100.0 ETH)', '0x5c6...21678 (100.0 ETH)', and '0x03C...D1Ff7 (100.0 ETH)'.

Switch back to the original wallet address(i.e the seller's wallet address) and within the transferOwnership function in the land transfer contract, enter:-

_landId: 1

_newOwner: [The Buyer's Address]

✓ LANDTRANSFER AT 0XA2E...DC

Balance: 0 ETH

TRANSFEROWNERSHIP

_landId: 1

_newOwner: 0xAb8483F64d9C6d1EcF9b849Ae67

Calldata

Parameters

transact

registry

Click on transact to complete the ownership transfer

0 ☐ Listen on all transactions

Filter with transaction hash or ad...

✕

transact to LandTransfer.transferOwnership pending ...

✓ [vm] from: 0x583...eddC4 to: LandTransfer.transferOwnership(uint256,address) 0xa2e...DCb97 value: 0 wei data: 0x295...35cb2 logs: 0 hash: 0xc18...d9a2b

Debug

status

1 Transaction mined and execution succeed

transaction hash

0xc188c22aff87cc55a3c65af8a5b87c469e02389c574b8ee4eb3c844087ad9a2b

block hash

0x3c0030a59908a64701260b4e6b9ead20a246afe02d20af27f0bbe6081d7bbe0d

block number

127

from

0x58380a6a701c568545dcfc803fc8875f56beddC4

to

LandTransfer.transferOwnership(uint256,address) 0xa2e9669fc58d05500af18aEd20dcf10A9e0DCb97

transaction cost

43957 gas

execution cost

22385 gas

output

0x

decoded input

{
"uint256 landId": "1",

Now, verify the ownership transfer by going to the LandRegistry contract. When we click on the getLand button, we see a different wallet address, indicating that the ownership transfer has taken place and is successfully verified.

The screenshot shows the Remix IDE interface. On the left, the 'REGISTERLAND' contract is displayed with fields for _id, _location, _area, and _owner. The 'getLand' button is highlighted. The main area shows the Solidity code for the LandRegistry contract. The right sidebar displays the transaction details, including the transaction cost, execution cost, output, and decoded input/output. The decoded input shows the parameters: uint256 _landId: '1' and address _newOwner: '0xab8483f64d9c6d1ecf9b849ae677d83315835cb2'.

Conclusion:

This experiment helped in understanding how smart contracts automate processes and execute securely on the Ethereum blockchain. Using Solidity and Remix IDE, the development, deployment, and interaction with smart contracts become simple and efficient, forming the foundation for decentralized applications.